Diffusion Model Lab 08 — Reflective Journal & Analysis

1. Understanding Diffusion

Going through both the forward and reverse diffusion helped solidify my understanding of how these models work. The forward process takes a clean MNIST digit and incrementally adds gaussian noise over a large number of small steps. What surprised me is how gradual the forward process really is. When I examined the images that came out of the reverse-process visualizations that I created, I could clearly see the model starting with nothing but noise and slowly whittling away at a digit's shape.

Another interesting observation was that the digit isn't recognizable right away. In almost all of my visualizations, I didn't know what the image was until about halfway through the denoising process (40–60%). Prior to this the image still has a high amount of noise, and what looks like a digit is only a faded shadow of its former self. This of course depended on the digit, with simpler digits like "1" or "7" appearing earlier and more complex digits like "5" or "8" showing up later.

Why add the noise in gradual steps — rather than all at once — now clicked for me. If the reverse process had to undo one large step of noise addition, it would have been impossible. Breaking up the corruption process into a large number of small steps allows the model to only learn how to denoise *a little* at each timestep. This makes the learning objective much more well behaved.

2. Model Architecture

It turned out that U-Net was ideally designed for diffusion models. I realized this only after observing how the model behaved during sampling. The encoder imposes a global structure to the image (what digit should appear), while the decoder fills in the details.

But the crucial ingredient is the skip connections. By integrating high-resolution features into the decoder, the U-Net eliminates the necessity of rebuilding intricate details solely from the compressed representations. Without skip connections, digits would always look blurry or morphed in some way. Since the decoder always has access to the original spatial information, it can generate sharp digits with clean edges.

Conditioning on the class was also key. In my model, the digit label is transformed into a one-hot vector, which gets embedded into a higher dimensional space. This embedding is injected into the U-Net, telling the model *which* digit to draw. During sampling, this conditioning vector directs the denoising trajectory towards the appropriate

class. I noticed this when requesting multiple digits. Even though the initial noise was randomized each time, the final digits were always correct

3. Training Analysis

The loss function tells us how closely the network is able to predict the added noise at each timestep. The lower the loss, the better your model understands the noise distribution and can reverse it. My training and validation losses decreased smoothly and didn't diverge far from each other. This indicated to me that my model was learning in a stable way, without overfitting.

Looking at generated images, I could see that the quality was increasing with training. In the beginning, the digits were blurry and sometimes even misinterpreted. After training for a while, they became crispier, centered, and more consistent. Finally, we see that our model produces nice and clean digits for all classes.

I also wanted to highlight what time embeddings are doing here. Our model needs to understand *how much noise* is present at each timestep so that it can condition its denoising behavior accordingly. If we're early in the process, we need a large correction. If we're late, we just need to make minor touch ups. Time embedding allows the model to know "where it is" in the denoising process.

4. CLIP Evaluation

Embedding CLIP into my evaluation pipeline allowed me to get another opinion on the quality of my generated digits. Specifically, for each sample, CLIP computed three scores: how likely it was to be a handwritten digit, how clear it looked, and how blurry/unreadable it looked...

CLIP gave highest scores to well-centered, high-contrast samples with clean strokes. The digits that scored highest were frequently “1”, “7”, and “9”. Conversely, “messier” digits tended to have lower clarity scores. I found that these scores lined up quite well with my own visual evaluations.

The trend that really stuck out to me was that simpler digits were easier for the model to produce. My guess is that digits with more curves/intersections have more subtle spatial dependencies that need to be learned in order to recreate them convincingly, which is difficult with a relatively small U-Net.

It may be possible to leverage CLIP scores during generation. For instance, I could generate k samples per digit and only keep the highest-scoring ones (“CLIP-guided selection”). More sophisticated techniques could even use CLIP feedback during the sampling loop itself to guide the model toward more canonical shapes.

5. Practical Applications

It's important to note that while this project focused on MNIST, the diffusion framework itself is very strong. Models trained with this technique can be applied to:

* Data augmentation (particularly nice for low-resource domains)

* Generating synthetic data for privacy-preserving training

* Creative uses such as style-controlled handwriting or artistic interpretations

* Restoration use cases such as denoising or inpainting

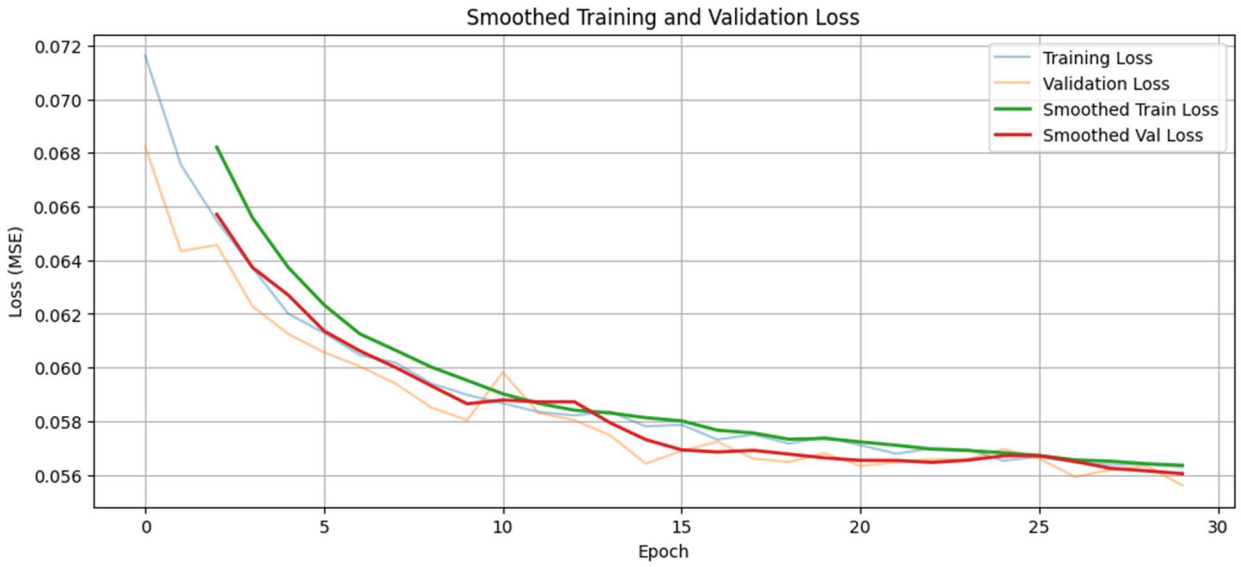
That being said, there are also limits to what the current model can do. Being trained only on low-resolution grayscale digits, it won't generalize to more interesting images. The U-Net itself is pretty small and I skipped techniques like EMA or classifier-free guidance which are present in cutting-edge diffusion models.

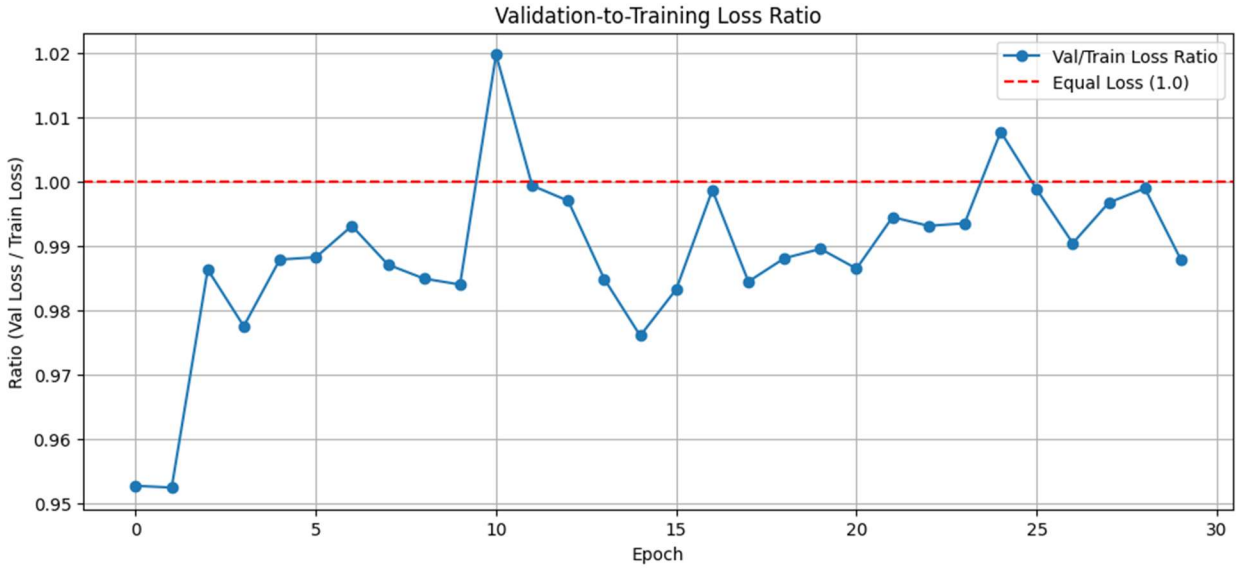
If I were to pick this project back up, there are 3 changes I would make:

* Add support for EMA weights - this typically results in sharper, higher-quality samples.

* Implement classifier-free guidance - this would allow me to tune the fidelity to a particular class as well as sample sharpness.

* Scale up architecture or dataset - Switching to CIFAR-10 images or a deeper U-Net would be interesting to see how much richer visual structure the model can handle.





Challenges & Troubleshooting Reflections

Building and debugging my diffusion model wasn't how I imagined working with machine learning models would be like. I imagined following recipes and having things work. Instead, every step along the way uncovered something new and every issue I ran into taught me a lesson.

Take U-Net for example. When you look at the schematic of the architecture, it all makes sense and looks fairly symmetric. But then you realize that something as trivial as mismatched channel dimensions will grind everything to a halt. In my case, my UpBlocks and DownBlocks were communicating using tensors with different channel dimensions. Debugging took me through multiple tensor shapes and connected me with the underlying architecture better than actually understanding the architecture did. Fixing layers to have the correct number of channels was one thing. Understanding skip connections was another. When everything worked, U-Net finally clicked for me.

Fast forward to sampling. After getting a batch of samples, I wanted to visualize the intermediate steps of the denoising process. But when I ran my visualize function, I got a shape-mismatch error related to my class conditioning mask. My mask was created using size of the one-hot vector (10), but during sampling, the model multiplies the mask with the class embedding which has 128 dimensions. After updating the mask dimension to match embedding, visualization worked! While the actual fix was trivial, it helped me understand how conditioning flows through the network.

One night, I must've stared at my training cell for minutes thinking the kernel was hung. For over 13 minutes, there was no output. Was something broken? Did I code something

incorrectly? Finally, I saw training start….and my cell finished a minute later. Diffusion models are slow. They have to because they iterate through so many timesteps. Lesson learned: never take progress bars and print statements for granted.

Then there were silent issues. Code cells that ran but didn't produce any output. Figures that didn't show up until I called plt.show(). Plots that wouldn't close after rendering, leaving giant blank spaces under them. Jupyter behaviors that seemed mysterious but were likely easy to fix with the right command or argument. These weren't exactly issues, but dealing with them kept me connected to the environment around the model.

The highlight of technical challenges for me was adding CLIP. While installing the package was easy, I had to tweak my generated MNIST digits in several ways to get them to work with the Vision Transformer:

1. Convert grayscale digits to RGB

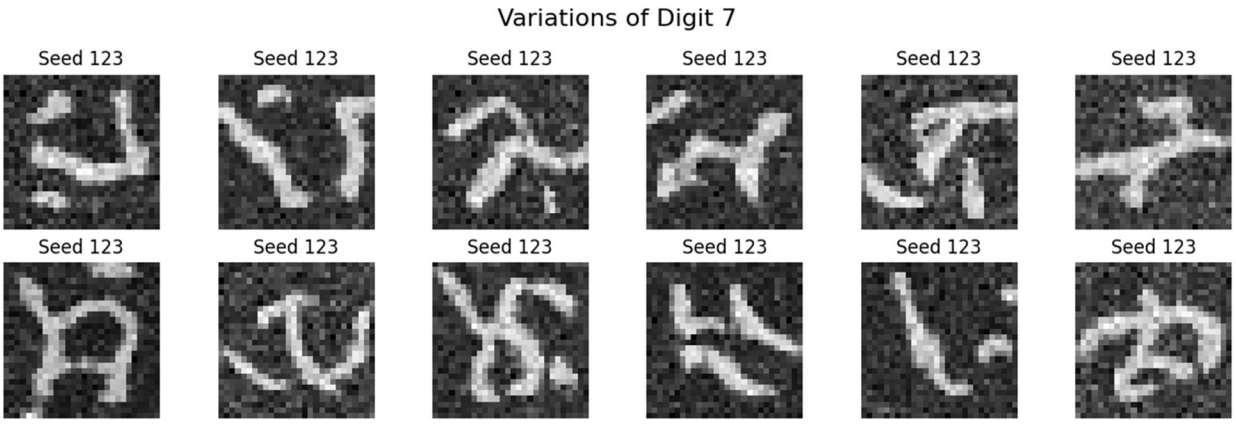
2. Resize to 224×224 (done automatically in batch transform)

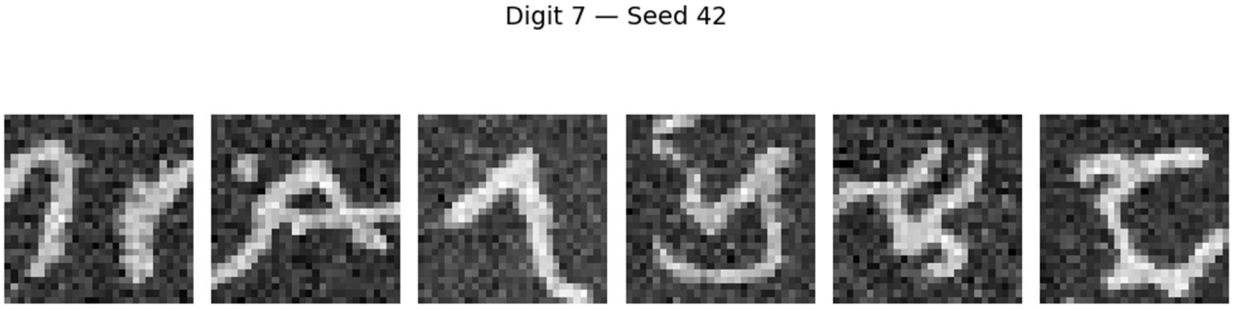
3. Normalize correctly

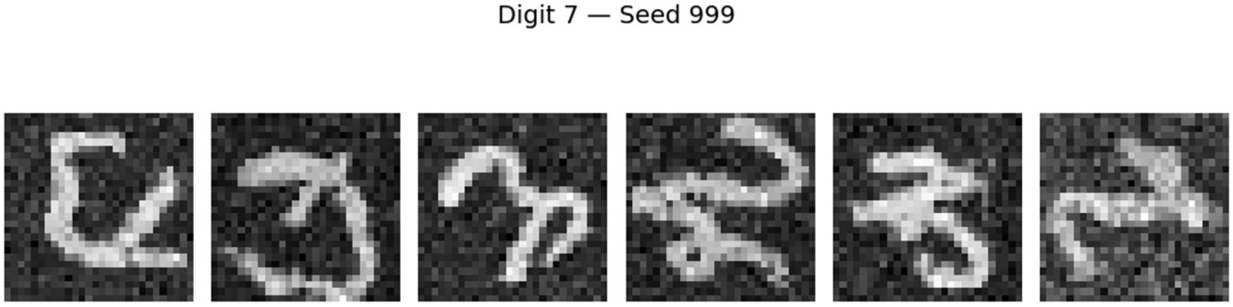
4. Clear GPU memory between loops to avoid eating up all VRAM and crashing

I ended up adding mixed-precision inference, chunking my eval function, and explicitly clearing memory after every loop. These might seem like simple quality-of-life improvements, but they made the entire pipeline feel more robust. More than just debugging, these tweaks forced me to better understand how CLIP worked and what it could do. With the kinks worked out, CLIP gave me a new way to look at samples and evaluate my model.

Even the model itself had something to teach me. I noticed that when I changed seeds, my digits would look drastically different from one another. At first, I thought there was something wrong with my sampler. But then I realized that because the seed controls the starting noise, your noise determines your entire trajectory of digit production. I added more seed tests to see the differences and in the end I was never truly pleased with any outputs. Satisfied that I got it working but not pleased.



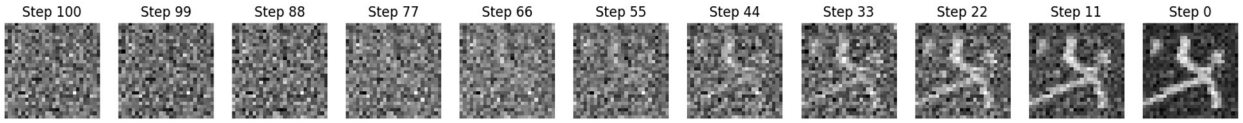


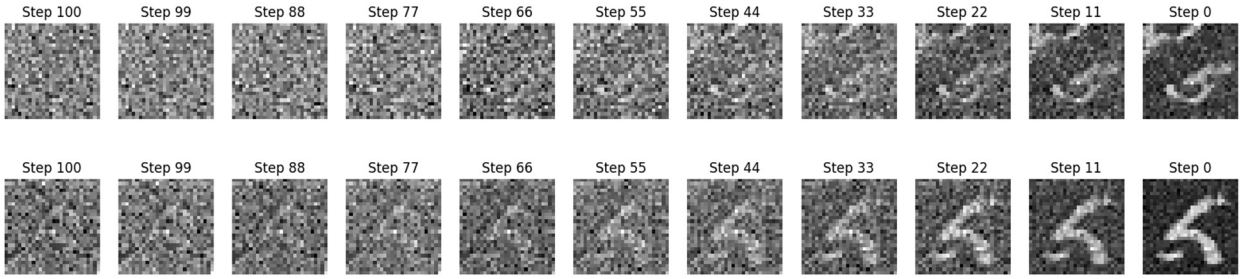


Trust me when I say I learned more from troubleshooting my model than I did from understanding how diffusion works. Every cryptic tensor exception, every silent code cell, every minute waiting for my model to train, felt like a hurdle. But as I looked back on my struggles piecing this notebook together, I realized that what I learned from troubleshooting was the best part of this entire project.

Step 7: Watching the Generation Process

Output images




Step 100
Step 99
Step 88
Step 77
Step 66
Step 55
Step 44
Step 33
Step 22
Step 11
Step 0
Step 100
Step 99
Step 88
Step 77
Step 66
Step 55
Step 44
Step 33
Step 22
Step 11
Step 0